《面向大学生成长路径的轻量级学习与项目管理系统设计与实现》

行，我们把流程细化成一个**用 Codex 驱动、但你始终掌控架构与质量**的开发节奏。我直接给你一套可执行的迭代路线 + 每阶段产物 + 检查点。

下面按 **8 周** 给（你说时间充足，做得更完整、更稳），你也可以压缩到 4–6 周照抄执行。

总体原则（Codex 最容易把你带偏的点先堵住）

1. **先定“数据模型 + API 合同”再写页面**：避免前后端反复返工。
2. **每周必须产出可演示版本**：随时可交付，不怕临时状况。
3. **Codex 写代码，人来写规范**：你负责架构、约束、验收用例；Codex 负责产出和搬砖。
4. **功能克制**：每个阶段只做“必须项 + 1 个加分项”。

Phase 0（0.5–1 天）产品与范围冻结

目标：不膨胀、不跑偏

产出物（文档/README 就行）：

- 用户故事（10 条以内）
- 模块清单：Project / Milestone / Task / Dashboard / WeeklyReview / AI
- 明确“不做”：多人协作、即时通讯、复杂权限、移动端原生

验收标准：

- 你能用 1 分钟讲清：核心工作流是什么、用户为什么会持续用它。

Phase 1（1–2 天）架构骨架 & 合同先行

目标：后端先能跑、接口先定死

1.1 后端骨架（SpringBoot）

- 基础工程：分层结构、统一返回体、全局异常、参数校验
- 认证：JWT 登录/注册/刷新（可选）
- 文档：Knife4j/Swagger
- 数据库：MySQL + Flyway/Liquibase（推荐 Flyway）

1.2 API 合同冻结（非常关键）

把核心 API 先写成 swagger（哪怕是假数据）：

- Auth：register/login/me
- Project：CRUD + complete/archive
- Task：CRUD + status/complete

- Milestone：CRUD + complete（先占位也行）
- Stats：overview（先返回 mock）
- AI：task-breakdown（先返回 mock）

☑ 检查点（必须通过）：

- `POST /login` 拿到 token
- `GET /me` 返回用户信息
- Swagger 页面能看到完整 API 列表

> Codex 用法：让它按你给的接口定义生成 Controller/DTO/Service 空壳，**但你先写好 DTO 字段与返回结构。**

Phase 2（3–5 天）MVP 闭环：项目 + 任务（先不追求好看）

目标：最小可交付版本

2.1 Project CRUD

- 新建/编辑/列表/详情
- 状态：进行中/完成/归档

2.2 Task CRUD（核心）

- 任务创建（挂 project，可选挂 milestone）
- 任务状态流转：待办→进行中→完成
- 逾期判断：`due_date < today && status != 完成`

2.3 权限隔离（必须做）

- 所有查询按 `user_id` 限制
- 404/403 处理清晰（不要泄露别人的资源）

☑ 检查点：

- 你可以开一个新账号，CRUD 自己的数据；换账号看不到对方数据。
- 任务完成时写入 `completed_at`，统计会用到。

Phase 3（3–5 天）Milestone（阶段）与项目进度

目标：从 ToDo 升级成“项目管理”，也是比赛加分点

功能：

- Milestone 列表 + 排序（`order_no`）
- milestone 完成率 = 其下任务完成率

- project 完成率 = 所有任务完成率（或里程碑加权，先简单）

☑ 检查点：

- Project 详情页能按 milestone 分组展示任务
- milestone 进度条能动（真实数据）

> Codex 用法：让它补齐 Service 里的进度计算与 DTO 拼装，但你要给它明确的口径（比如“完成率=完成数/总数，空时返回0”）。

Phase 4（2–4 天）Dashboard（统计可视化，控制在 4 张图以内）

目标：让评委一眼看到“工作量”和“价值”

建议只做这 3–4 个：

1. 近 7 天每天完成任务数
2. 当前进行中项目数、逾期任务数、今日到期任务数（卡片）
3. 项目完成率排行（Top N） 4）（可选）周热力图

后端：

- `GET /stats/overview` 一次返回全部，避免前端多次请求

☑ 检查点：

- 用真实数据驱动，空数据也能正常展示（0 值、空数组）
- SQL 性能 OK（索引：task(project_id,status,due_date,completed_at)）

Phase 5（2–4 天）Weekly Review（周总结：成长反馈闭环）

目标：让工具“可持续使用”，而不是一次性

功能：

- 选择某周 → 自动汇总：完成任务数、完成项目数、最常做的项目
- 用户输入本周反思（content）
- 历史周总结列表

☑ 检查点：

- 周起止逻辑固定（建议周一到周日）
- 支持导出 Markdown（可选加分）

Phase 6（3–6 天）AI（最后上，且只做 2 个点）

目标：AI 成为“效率增强器”，而不是不稳定核心

6.1 任务拆解（重点）

- 输入：目标/项目描述/周期
- 输出：milestones + tasks（草稿）
- 必须：用户确认后再落库（apply）

6.2 周总结润色（低风险）

- 输入：本周数据 + 用户反思
- 输出：100–200 字总结 + 下周建议 3 条

☑ 检查点：

- AI 输出结构稳定（JSON schema），失败有兜底提示
- 有 `ai_generation` 记录可查（比赛材料很好写）

> 你比赛工具限制要注意：通知里要求 AI 工具需在规定列表内或自研工具，且生成内容要在报名表注明技术路径 。你用 Codex 写代码没问题，但“作品中的 AI 生成内容/能力点”建议落在这些允许的工具上（比如文心/智谱等），写材料更稳。

Phase 7（2–4 天）工程化收尾：部署、演示、材料

目标：把分拿到手

- 一键部署：后端（Docker 或 jar）+ 前端（nginx）
- 演示脚本：10 分钟内讲清楚 + 2 分钟内展示亮点（Dashboard/AI）
- 风险排查：
  - 空数据、异常输入、token 过期
  - 逾期计算、时区/日期边界
  - 幂等（重复点击完成按钮不崩）

☑ 产出物清单（比赛友好）：

- 可访问网址
- 演示视频（≤10分钟）
- 使用手册（截图）
- 技术文档（架构图 + 数据表 + API）
- AI 使用说明（工具、提示词策略、人工确认机制）

你用 Codex 的“高产提示词模板”（直接复制用）

你每次让 Codex 干活，按这个结构喂：

- 背景：这是一个 SpringBoot 项目管理系统
- 约束：分层、DTO/Entity 分离、统一返回体、按 user_id 权限隔离
- 输入：给出 DTO 字段、接口路径、业务口径
- 输出：要求生成哪些文件 + 单元测试（至少 service 层）
- 验收：列 3 条你要怎么验证它写得对

这样 Codex 不容易乱发挥。